

**LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI
OBJEK
PRAKTIKUM 3
RELASI ANTAR OBJEK DAN TRANSAKSI REKENING**



OLEH

Nama : Lexi Mulia Yunaspi

NIM : 2511531006

Kelas : B

MATA KULIAH PEMROGRAMAN BERORIENTASI OBJEK

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM

ASISTEN PRAKTIKUM : EZZA ADDINI

**Program Studi Informatika
Fakultas Teknologi Informasi**

Universitas Andalas

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	3
1.3 Manfaat.....	3
BAB II PEMBAHASAN.....	4
A. LATIHAN.....	6
2.1 Persiapan Project dan Package.....	6
2.2 Class Transaksi.....	8
2.3 Relasi Class Rekening dan Transaksi.....	10
2.4 Pencatatan Setor Tunai.....	12
B. TUGAS.....	13
2.5 Tugas 1 – Integrasi Rekam Jejak Penarikan.....	13
2.6 Tugas 2 – Fitur Cetak Mutasi Rekening.....	15
2.7 Penambahan Menu 6 pada Main.....	16
2.8 Integrasi Program pada Class Main.....	15
C. HASIL PENGUJIAN.....	16
2.9 Hasil Pengujian Praktikum.....	16
2.10 Rangkuman Latihan dan Tugas.....	24
BAB III PENUTUP.....	24
3.1 Kesimpulan.....	24
3.2 Saran.....	24
DAFTAR PUSTAKA.....	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek menggunakan class dan object sebagai dasar penyusunan program. Pada praktikum ini program perbankan dikembangkan dengan membuat class Transaksi yang berhubungan dengan class Rekening. Satu rekening dapat memiliki banyak transaksi sehingga digunakan ArrayList untuk menyimpan riwayat.

Program juga memiliki fitur setor tunai, tarik tunai, cek informasi rekening, ganti akun, dan cetak mutasi. Class Main digunakan untuk mengatur input pengguna dan menu program. Ketiga class tersebut bekerja bersama dalam satu program perbankan mini.

1.2 Tujuan

Tujuan praktikum ini adalah memahami hubungan antar object pada Java, membuat class Transaksi, menghubungkannya dengan class Rekening, dan menggunakan ArrayList untuk menyimpan banyak transaksi. Praktikum juga bertujuan menerapkan validasi dan fitur cetak mutasi.

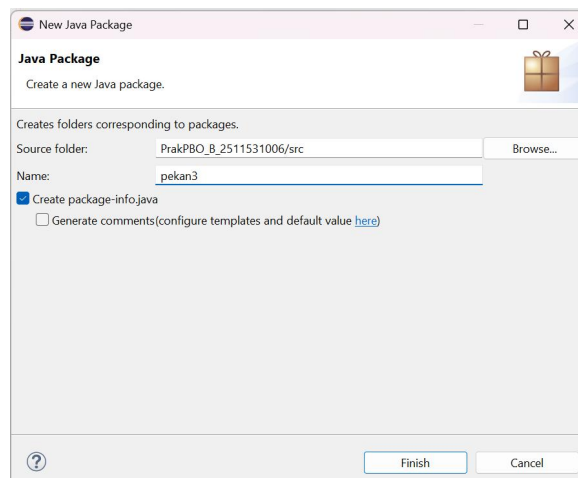
1.3 Manfaat

Praktikum ini membantu memahami constructor, object, ArrayList, percabangan, perulangan, switch-case, Scanner, dan pemanggilan method antar class. Selain itu, mahasiswa dapat memahami bagaimana data rekening dan transaksi dikelola dalam program Java.

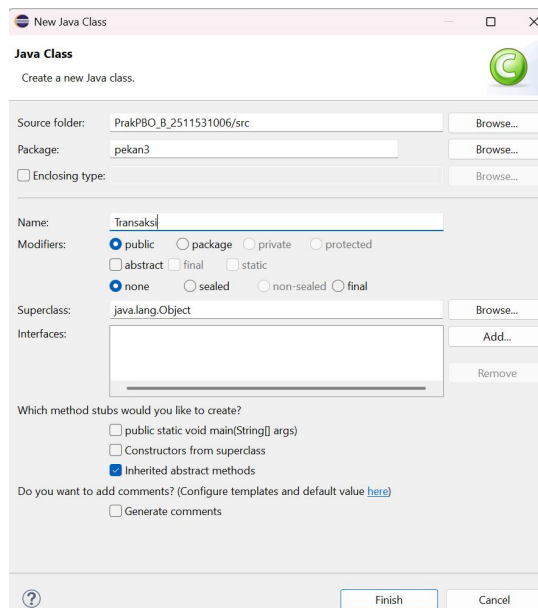
BAB II

PEMBAHASAN

Gambar di bawah menunjukkan proses pembuatan package baru pada Eclipse. Source folder diarahkan ke project PrakPBO_B_2511531006/src dan nama package diisi pekan3. Package ini digunakan untuk menempatkan class Praktikum 3 agar terpisah dari package praktikum sebelumnya.

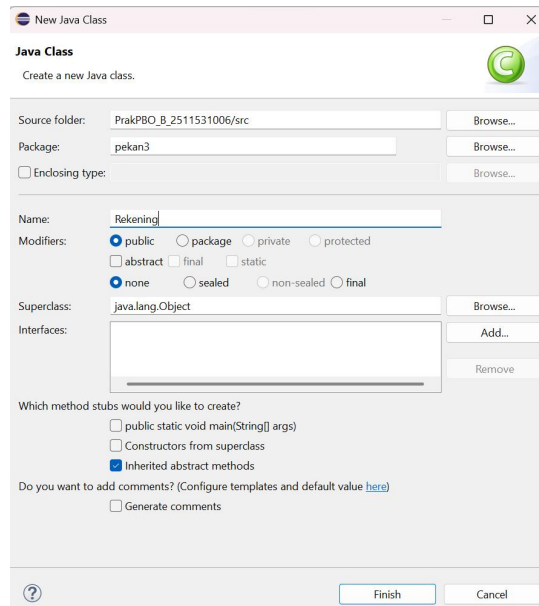


Gambar 2.1 Pembuatan package pekan3



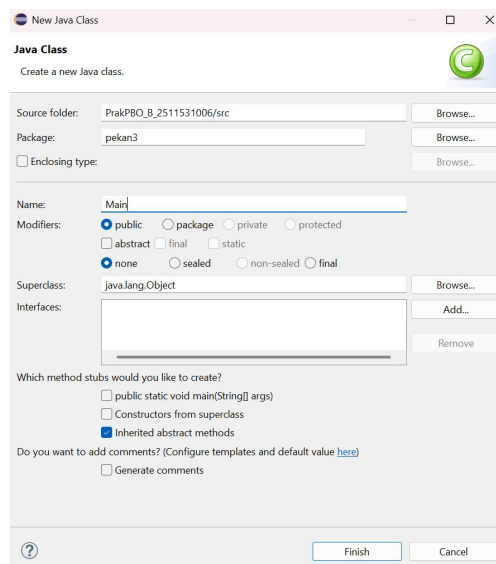
Gambar 2.2 Pembuatan class Transaksi

Gambar menunjukkan proses pembuatan class Transaksi di dalam package pekan3. Nama class diisi Transaksi dan modifier public dipilih agar dapat digunakan class lain. Method main tidak dipilih karena Transaksi bukan class utama.



Gambar 2.3 Pembuatan class Rekening

Gambar menunjukkan proses pembuatan class Rekening pada package pekan3. Class ini digunakan untuk menyimpan data rekening dan riwayat transaksi. Method main tidak dipilih karena program utama berada pada class Main.



Gambar 2.4 Pembuatan class Main

Gambar menunjukkan proses pembuatan class Main pada package pekan3. Opsi public static void main(String[] args) dipilih karena Main menjadi class utama. Menu dan proses interaksi pengguna ditempatkan pada method main.

A. LATIHAN

2.1 Persiapan Project dan Package

Praktikum 3 menggunakan project PrakPBO_B_2511531006 dan membuat package baru bernama pekan3. Package ini digunakan untuk menyimpan class Transaksi, Rekening, dan Main. Langkahnya dibuat sama seperti susunan pada Laprak 2, tetapi nama package disesuaikan dengan Praktikum 3.

a. Package pada class

```
1 package pekan3;  
2
```

Baris package menentukan lokasi class dalam struktur project Java. Package pekan3 memisahkan class Praktikum 3 dari package praktikum sebelumnya. Ketiga class utama ditempatkan dalam package yang sama agar dapat saling digunakan. Pembagian ini membuat struktur project lebih rapi dan mudah dipahami.

b. Pembagian fungsi class

Class Transaksi menyimpan data satu transaksi seperti ID, jenis, dan nominal. Class Rekening menyimpan data rekening serta kumpulan riwayat transaksi. Class Main menerima input pengguna dan mengatur menu program. Ketiga class tersebut bekerja bersama membentuk sistem perbankan mini.

2.2 Class Transaksi

Class Transaksi digunakan untuk mewakili satu transaksi yang terjadi pada rekening. Seluruh bagian class dijelaskan mulai dari package, atribut, constructor, getter, sampai method cetakDetail().

a. Package dan deklarasi class

```
1 package pekan3;  
2  
3 public class Transaksi {  
4
```

Baris package menempatkan class Transaksi pada package pekan3. public class Transaksi mendeklarasikan class yang dapat digunakan class lain. Kurung kurawal membuka bagian isi class Transaksi. Class ini menjadi model data untuk setiap transaksi yang dibuat.

b. Atribut transaksi

```
5 private String idTransaksi;  
6 private String jenis;  
7 private double nominal;  
8
```

idTransaksi digunakan sebagai identitas dari setiap transaksi. jenis digunakan untuk membedakan transaksi Kredit dan Debit. nominal menyimpan jumlah uang yang terlibat dalam transaksi. Keyword private menjaga atribut agar tidak diakses langsung dari class lain.

c. Constructor Transaksi

```
public Transaksi(String idTransaksi, String jenis, double nominal)  
{  
    this.idTransaksi = idTransaksi;  
    this.jenis = jenis;  
    this.nominal = nominal;  
}
```

Constructor digunakan untuk memberikan nilai awal saat object Transaksi dibuat. Parameter constructor menerima ID, jenis, dan nominal transaksi. Keyword `this` menunjukkan atribut yang dimiliki oleh object. Nilai parameter kemudian disimpan ke dalam atribut masing-masing.

d. Getter Transaksi

```
public String getIdTransaksi() {return idTransaksi; }  
public String getJenis() {return jenis; }  
public double getNominal() {return nominal; }
```

Getter digunakan untuk mengambil sebuah nilai atribut yang bersifat private. `getIdTransaksi()` mengembalikan ID transaksi yang tersimpan. `getJenis()` dan `getNominal()` mengembalikan jenis serta nominal transaksi. Method getter memungkinkan class lain membaca data tanpa mengakses atribut langsung.

e. Method cetakDetail()

```
9 public void cetakDetail() {  
0     System.out.println("ID Transaksi : " + idTransaksi+ " | Jenis : " + jenis+ " | Nominal : Rp" + nominal);  
1 }  
2 }
```

Method `cetakDetail()` digunakan untuk menampilkan informasi di sebuah transaksi. `System.out.println` menggabungkan ID, jenis, dan nominal menggunakan operator `+`. Informasi transaksi ditampilkan dalam satu baris pada Console. Method ini nantinya dipanggil saat riwayat transaksi dicetak.

2.3 Relasi Class Rekening dan Transaksi

Class Rekening mempunyai hubungan dengan class Transaksi karena satu rekening dapat mempunyai banyak transaksi. Hubungan tersebut dibuat menggunakan `ArrayList<Transaksi>`. Bagian berikut menjelaskan seluruh kode Rekening yang berkaitan dengan data rekening dan transaksi.

a. Import dan deklarasi class

```
1 package pekan3;  
2  
3 import java.util.ArrayList;  
4  
5 public class Rekening {
```

package pekan3 menempatkan class Rekening pada package yang sama. ArrayList diimpor untuk menyimpan banyak object Transaksi. public class Rekening mendeklarasikan class yang mengelola data rekening. Class ini menjadi tempat utama pengelolaan saldo dan riwayat transaksi.

b. Atribut rekening

```
7 private String nomorRekening;  
8 private String namaPemilik;  
9 private double saldo;  
10 private String pin;
```

nomorRekening menyimpan nomor identitas rekening pengguna. namaPemilik menyimpan nama pemilik rekening. saldo menyimpan jumlah uang yang dimiliki rekening. pin menyimpan PIN yang digunakan untuk otentikasi.

c. Riwayat transaksi

```
private ArrayList<Transaksi> riwayatTransaksi = new ArrayList<Transaksi>();
```

riwayatTransaksi merupakan ArrayList yang berisi object Transaksi. Setiap rekening mempunyai tempat sendiri untuk menyimpan riwayat. Object Transaksi dapat ditambahkan setelah transaksi berhasil dilakukan. Bagian ini menunjukkan hubungan satu Rekening dengan banyak Transaksi.

d. Constructor Rekening

```
public Rekening(String nomor, String nama, double saldoAwal, String pin) {  
    nomorRekening = nomor;  
    namaPemilik = nama;  
    saldo = saldoAwal;  
    this.pin = pin;  
    System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);  
}
```

Constructor menerima nomor, nama, saldo awal, dan PIN dari pengguna. Data tersebut disimpan ke atribut object Rekening yang baru dibuat. `System.out.println` menampilkan informasi bahwa rekening berhasil dibuat. Constructor dijalankan ketika class Main membuat object Rekening.

e. Getter Rekening

```
public String getNomorRekening() {return nomorRekening;}  
public String getNamaPemilik() {return namaPemilik;}  
public double getSaldo() {return saldo;}
```

`getNomorRekening()` digunakan untuk mengambil nomor rekening. `getNamaPemilik()` digunakan untuk mengambil nama pemilik rekening. `getSaldo()` digunakan untuk membaca saldo yang tersimpan. Ketiga getter membantu class Main membaca data private Rekening.

2.4 Pencatatan Setor Tunai

Pada bagian ini, pencatatan transaksi setor tunai dijelaskan berdasarkan perubahan pada class Rekening. Method `setorTunai()` memeriksa agar nominal setoran lebih dari nol sebelum saldo dan riwayat transaksi diperbarui. Jika setoran valid, saldo bertambah dan object Transaksi Kredit dimasukkan ke `riwayatTransaksi`. Dengan demikian, transaksi setor yang berhasil dapat tersimpan dan ditampilkan kembali pada fitur mutasi.

B. TUGAS

2.5 Tugas 1 – Integrasi Rekam Jejak Penarikan

Tugas 1 meminta setiap penarikan yang berhasil mempunyai rekam jejak transaksi. Penarikan gagal tidak boleh dicatat. Karena itu object Transaksi Debit dibuat setelah pemeriksaan saldo dan batas minimum berhasil dilewati.

a. Method tarikTunai dan pemeriksaan saldo

```
public void tarikTunai(double nominal) {  
    if (nominal > saldo) {  
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp"+ saldo);
```

Method tarikTunai menerima nominal uang yang ingin ditarik. Kondisi pertama memeriksa apakah nominal melebihi saldo. Jika saldo tidak mencukupi, transaksi dihentikan dan saldo tetap. Pada kondisi ini object Transaksi tidak dibuat dan tidak masuk riwayat.

b. Pemeriksaan minimum penarikan

```
} else if (nominal < 10000) {  
    System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
```

Kondisi else if memeriksa batas minimum penarikan. Nominal kurang dari Rp10.000 dianggap tidak memenuhi aturan program. Saldo tidak dikurangi ketika kondisi tersebut terpenuhi. Transaksi gagal juga tidak ditambahkan ke riwayat.

c. Penarikan berhasil

```
} else {  
    saldo -= nominal;
```

Blok else dijalankan jika saldo mencukupi dan nominal memenuhi batas minimum. Operator -= mengurangi saldo sesuai nominal penarikan. Perubahan saldo dilakukan sebelum transaksi Debit dibuat. Setelah saldo berhasil berubah, proses pencatatan dapat dilanjutkan.

d. Membuat transaksi Debit

```
saldo -= nominal;  
Transaksi transaksiBaru = new Transaksi("TRX-T-" + (riwayatTransaksi.size() + 1), "Debit", nominal);
```

Object Transaksi dibuat untuk merekam penarikan yang berhasil.ID transaksi menggunakan awalan TRX-T- dan nomor urut riwayat.Jenis transaksi diisi Debit karena saldo rekening berkurang.Nominal yang disimpan sama dengan jumlah uang yang ditarik.

e. Menyimpan transaksi penarikan

```
riwayatTransaksi.add(transaksiBaru);
```

Object transaksiBaru dimasukkan ke ArrayList riwayatTransaksi.Penarikan berhasil kemudian dapat dilihat kembali melalui mutasi.Penarikan yang gagal tidak mencapai baris add tersebut.Hal ini membuat riwayat hanya berisi transaksi yang berhasil.

f. Method cekInformasi()

```
public void cekInformasi() {  
    System.out.println("--- INFO REKENING ---");  
    System.out.println("No. Rekening : " + nomorRekening);  
    System.out.println("Nama Pemilik : " + namaPemilik);  
    System.out.println("Saldo Akhir : Rp" + saldo);  
}
```

Method cekInformasi() digunakan untuk menampilkan data rekening aktif.Program mencetak nomor rekening dan nama pemilik.Program juga mencetak saldo akhir yang tersimpan pada object.Method ini dipanggil dari menu Cek Informasi Rekening pada Main.

2.6 Tugas 2 – Fitur Cetak Mutasi Rekening

Tugas 2 menambahkan method cetakMutasi pada class Rekening. Method ini membaca seluruh object Transaksi yang tersimpan dalam ArrayList. Jika belum ada transaksi, program menampilkan pesan khusus.

a. Method cetakMutasi

```
public void cetakMutasi() {  
    System.out.println("--- MUTASI REKENING ---");
```

Method `cetakMutasi()` digunakan untuk menampilkan riwayat transaksi rekening. Baris `println` pertama memberikan judul pada tampilan mutasi. Riwayat yang ditampilkan berasal dari `ArrayList` milik rekening aktif. Method ini dijalankan setelah pengguna melewati pemeriksaan PIN.

b. Memeriksa riwayat kosong

```
if (riwayatTransaksi.isEmpty()) {  
    System.out.println("Belum ada transaksi pada rekening ini");
```

`isEmpty()` digunakan untuk memeriksa apakah `ArrayList` tidak memiliki data. Jika riwayat kosong, program tidak menjalankan perulangan transaksi. Program menampilkan pesan bahwa belum ada transaksi pada rekening. Kondisi ini penting untuk rekening baru yang belum melakukan transaksi.

c. Menampilkan seluruh transaksi

```
    } else {  
        for (Transaksi transaksi : riwayatTransaksi) {  
            transaksi.cetakDetail();  
        }  
    }
```

For-each mengambil object `Transaksi` satu per satu dari riwayat. Variabel transaksi mewakili object yang sedang diproses. Method `cetakDetail()` dipanggil untuk menampilkan detail transaksi. Semua transaksi dalam `ArrayList` akhirnya ditampilkan secara berurutan.

d. Penutup output

```
75  
76  
77    }  
    System.out.println();
```

Baris ini memberikan baris kosong setelah daftar mutasi selesai ditampilkan. Tujuannya agar tampilan Console tidak terlalu rapat dengan menu berikutnya. Kurung kurawal menutup method cetakMutasi setelah seluruh proses selesai. Method kemudian kembali ke bagian program yang memanggilnya.

2.7 Penambahan Menu 6 pada Main

Agar fitur cetak mutasi dapat digunakan, menu utama pada class Main mempunyai pilihan nomor 6. Ketika dipilih, program memeriksa akun aktif, meminta PIN, dan memanggil cetakMutasi jika PIN benar.

a. Menambahkan menu

```
System.out.println("6. Cetak Mutasi (Riwayat)");  
System.out.println("7. Keluar");
```

Baris tersebut menampilkan pilihan Cetak Mutasi pada menu utama. Nomor 6 digunakan untuk membuka riwayat transaksi rekening aktif. Nomor 7 digunakan untuk menghentikan program. Kedua pilihan tersebut dijalankan melalui switch pada Main.

b. Case 6

```
case 6:  
    if (akunAktif == null) {  
        System.out.println("Error: Anda belum membuka rekening!");
```

case 6 dijalankan ketika pengguna memilih menu Cetak Mutasi. Program terlebih dahulu memeriksa apakah akunAktif masih null. Jika belum ada rekening, program menampilkan pesan kesalahan. Jika ada rekening, proses dilanjutkan ke pemeriksaan PIN.

c. Memanggil cetakMutasi

```
    } else {  
        System.out.print("Masukkan PIN: ");  
        String pinMutasi = input.nextLine();  
  
        if (akunAktif.otentikasi(pinMutasi)) {  
            akunAktif.cetakMutasi();
```

Program meminta PIN sebelum riwayat transaksi ditampilkan. Method `otentikasi()` membandingkan PIN input dengan PIN rekening. Jika PIN benar, method `cetakMutasi()` dipanggil pada akun aktif. Jika PIN salah, program menolak akses dan tidak mencetak mutasi.

2.8 Integrasi Program pada Class Main

Setelah fitur tugas selesai dibuat pada class Rekening, perubahan tersebut dihubungkan ke menu pada class Main. Bagian ini menjelaskan kode Main secara bertahap pada bagian yang berkaitan dengan input PIN, autentikasi, dan menu fitur baru.

a. Input PIN saat membuka rekening

```
System.out.print("Masukkan PIN (6 digit): ");  
String pin = input.nextLine();
```

Program meminta pengguna memasukkan PIN ketika memilih menu pembukaan rekening. PIN disimpan ke dalam variabel `pin` sebelum object Rekening dibuat. Nilai tersebut kemudian diteruskan ke constructor Rekening bersama data rekening lainnya. Dengan cara ini setiap rekening yang dibuat memiliki PIN untuk autentikasi.

b. Case 3 – Tarik tunai dengan autentikasi

```
String pinTarik = input.nextLine();  
if (akunAktif.otentikasi(pinTarik)) {
```

Pada case tarik tunai, program meminta PIN dari pengguna sebelum proses penarikan. PIN tersebut diperiksa menggunakan method `otentikasi()` milik `akunAktif`. Jika hasilnya `true`, pengguna dapat melanjutkan dengan memasukkan nominal penarikan. Jika PIN salah, program menampilkan pesan akses ditolak dan transaksi tidak diproses.

c. Case 6 – Cetak mutasi dengan autentikasi

```
String pinMutasi = input.nextLine();  
if (akunAktif.otentikasi(pinMutasi)) {
```

Case 6 meminta PIN sebelum riwayat transaksi ditampilkan. Program menggunakan `otentikasi()` untuk memastikan PIN sesuai dengan rekening aktif. Jika autentikasi berhasil, method `cetakMutasi()` dipanggil untuk menampilkan riwayat. Jika PIN salah, program tidak melanjutkan proses pencetakan mutasi.

d. Menu 6 dan menu keluar

```
System.out.println("6. Cetak Mutasi (Riwayat)");  
System.out.println("7. Keluar");
```

Menu 6 ditambahkan agar pengguna dapat mengakses fitur cetak mutasi. Menu 7 digunakan untuk mengakhiri perulangan menu program. Saat pilihan 7 dijalankan, nilai `isRunning` diubah menjadi `false`. Perubahan tersebut membuat program keluar dari perulangan utama.

C. HASIL PENGUJIAN

2.9 Hasil Pengujian Praktikum

Pengujian dilakukan untuk memastikan menu dan fitur pada ketiga class dapat bekerja sesuai dengan kode.

a. Pengujian 1 – Membuka Rekening Baru

Menu 1 dipilih untuk membuat rekening baru. Nomor rekening, nama, saldo awal, dan PIN 6 digit berhasil dimasukkan. Program menampilkan bahwa rekening berhasil dibuat dengan saldo Rp5.000.000. Hasil ini menunjukkan proses pembukaan rekening berjalan dengan baik.

```

Console X
Main (3) [Java Application] C:\Users\Hype AMD\p2\pool\plugins\org.eclipse.justj.openjdk.hot
==SISTEM PERBANKAN MINI==

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 1
Masukkan No Rekening: 1234567
Masukkan Nama Pemilik: lexi
Masukkan Saldo Awal: 5000000
Masukkan PIN (6 digit): 123456
Rekening atas nama lexi berhasil dibuat dengan saldo Rp5000000.0

```

b. Pengujian 2 – Setor Tunai

```

Main (3) [Java Application] C:\Users\Hype AMD\p2\pool\plugins\org.eclipse.justj.openjdk.hot
==SISTEM PERBANKAN MINI==

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 2
Error: Mohon maaf, Anda belum memiliki nomor rekening!

```

Menu 2 dipilih untuk melakukan setor tunai. Nominal sebesar Rp2.000.000 dimasukkan ke rekening. Program menampilkan saldo bertambah menjadi Rp7.000.000. Hasil ini menunjukkan proses setor tunai berhasil dilakukan.

c. Pengujian 3 – Tarik Tunai Berhasil

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik: 3000000
Tarik tunai Rp3000000.0 berhasil. Saldo saat ini: Rp4000000.0

```

Menu 3 dipilih dan PIN yang benar yaitu 123456 dimasukkan. Nominal penarikan sebesar Rp3.000.000 dimasukkan. Program menampilkan penarikan berhasil dan saldo menjadi Rp4.000.000. Hasil ini menunjukkan tarik tunai dengan PIN benar berhasil.

d. Pengujian 4 – Cek Informasi Rekening

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 1234567
Nama Pemilik : lexi
Saldo Akhir  : Rp4000000.0

```

Menu 4 dipilih untuk melihat informasi rekening yang aktif. Program menampilkan nomor rekening 1234567 dan nama pemilik lexi. Saldo akhir yang ditampilkan adalah Rp4.000.000. Hasil ini menunjukkan informasi rekening dapat ditampilkan dengan benar.

e. Pengujian 5 – Ganti Akun

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 5
Masukkan nomor rekening: 1234567
Akun berhasil diganti.

```

Menu 5 dipilih untuk mengganti rekening yang sedang digunakan. Nomor rekening 1234567 dimasukkan sebagai rekening tujuan. Program menampilkan pesan bahwa akun berhasil diganti. Hasil ini menunjukkan proses pergantian akun berhasil dilakukan.

f. Pengujian 6 – Cetak Mutasi

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 6
Masukkan PIN: 123456
--- MUTASI REKENING ---
ID Transaksi : TRX-S-1 | Jenis : Kredit | Nominal : Rp2000000.0
ID Transaksi : TRX-T-2 | Jenis : Debit | Nominal : Rp3000000.0

```

Menu 6 dipilih dan PIN 123456 dimasukkan dengan benar. Program menampilkan daftar transaksi pada rekening aktif. Terlihat transaksi kredit Rp2.000.000 dan debit Rp3.000.000. Hasil ini menunjukkan riwayat transaksi berhasil ditampilkan.

g. Pengujian 7 – PIN Salah

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 3
Masukkan PIN: 1234567
Akses Ditolak: PIN yang Anda masukkan salah!

```

Menu 3 dipilih dan PIN yang dimasukkan adalah 1234567. PIN tersebut tidak sesuai dengan PIN rekening yang tersimpan. Program menampilkan pesan “Akses Ditolak: PIN yang Anda masukkan salah!”. Hasil ini menunjukkan sistem berhasil menolak PIN yang salah.

h. Pengujian 8 – Tarik Melebihi Saldo

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik: 6000000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp4000000.0

```

Menu 3 dipilih dengan PIN yang benar. Nominal penarikan sebesar Rp6.000.000 dimasukkan. Program menolak transaksi karena saldo hanya Rp4.000.000. Hasil ini menunjukkan penarikan melebihi saldo tidak dapat dilakukan.

i. Pengujian 9 – Tarik di Bawah Rp10.000

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik: 2
Transaksi Gagal : Minimal nominal penarikan 10.000

```

Menu 3 dipilih dan PIN yang benar dimasukkan. Nominal penarikan sebesar Rp2 dimasukkan. Program menampilkan pesan bahwa minimal penarikan adalah Rp10.000. Hasil ini menunjukkan nominal yang terlalu kecil berhasil ditolak.

j. Pengujian 10 – Setor Nol atau Negatif

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 2
Masukkan nominal setor: 0
Gagal: Nominal setor harus lebih dari 0!

```

Menu 2 dipilih untuk melakukan setor tunai. Nominal setor sebesar Rp0 dimasukkan ke dalam program. Program menampilkan pesan “Gagal: Nominal setor harus lebih dari 0!”. Hasil ini menunjukkan setor dengan nominal nol berhasil ditolak.

k. Pengujian 11 – Rekening Tanpa Transaksi

```
Main (3) [Java Application] C:\Users\Hype AMD\p2\pool\plugins\org.eclipse.justj.openjdk.ho
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 1
Masukkan No Rekening: 789000
Masukkan Nama Pemilik: caca
Masukkan Saldo Awal: 2000000
Masukkan PIN (6 digit): 123456
Rekening atas nama caca berhasil dibuat dengan saldo Rp2000000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 6
Masukkan PIN: 123456
|--- MUTASI REKENING ---
Belum ada transaksi pada rekening ini
```

Rekening baru dibuat dengan saldo awal Rp2.000.000. Menu 6 dipilih dan PIN 123456 dimasukkan dengan benar. Program menampilkan pesan “Belum ada transaksi pada rekening ini”. Hasil ini menunjukkan mutasi kosong dapat ditampilkan dengan benar.

l. Pengujian 12 – Menu Tidak Valid

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 8
Pilihan tidak valid!

```

Program dijalankan lalu pilihan menu 8 dimasukkan. Pilihan tersebut tidak tersedia pada menu utama. Program menampilkan pesan “Pilihan tidak valid!”. Hasil ini menunjukkan pilihan di luar menu berhasil ditolak.

m. Pengujian 13 – Tanpa Rekening Aktif

```

main.cpp:4:1: warning: braces around conditional operator are not needed [warning (group)]
==SISTEM PERBANKAN MINI==

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 2
Error: Mohon maaf, Anda belum memiliki nomor rekening!

```

Program dijalankan tanpa membuat rekening terlebih dahulu. Menu 2 dipilih untuk melakukan setor tunai. Program menampilkan pesan “Anda belum memiliki nomor rekening!”. Hasil ini menunjukkan program dapat menolak transaksi tanpa rekening aktif.

Praktikum ini menggunakan tiga class yang saling berhubungan, yaitu Transaksi, Rekening, dan Main. Transaksi menyimpan detail transaksi, Rekening mengelola data rekening dan riwayat, sedangkan Main mengatur input serta menu program.

Latihan membahas pembuatan package dan class, hubungan Rekening dengan Transaksi, serta pencatatan setor tunai. Tugas membahas rekam jejak penarikan, fitur cetak mutasi, penambahan menu 6, dan integrasi seluruh program.

Validasi juga diterapkan agar transaksi gagal tidak masuk ke riwayat. Dengan demikian program tidak hanya mengubah saldo, tetapi juga menyimpan transaksi yang berhasil dan dapat menampilkannya kembali.

BAB III

PENUTUP

3.1 Kesimpulan

Praktikum 3 menunjukkan penerapan relasi antar object melalui tiga class, yaitu Transaksi, Rekening, dan Main. Class Transaksi menyimpan data transaksi, Rekening mengelola data rekening dan riwayat, sedangkan Main mengatur jalannya program.

Seluruh kode pada ketiga class digunakan bersama untuk membuat sistem perbankan mini. Program dapat membuat rekening, melakukan setor dan tarik tunai, mengecek informasi, mengganti akun, dan mencetak mutasi dengan validasi yang sesuai.

3.2 Saran

Program dapat dikembangkan dengan menambahkan penyimpanan data secara permanen, tanggal transaksi, validasi PIN yang lebih lengkap, serta fitur pengelolaan rekening lainnya. Pengembangan tersebut dapat membuat program lebih lengkap dan mendekati aplikasi perbankan sederhana.

DAFTAR PUSTAKA

Modul Praktikum Pemrograman Berorientasi Objek. Praktikum 3. Program Perbankan Mini.
Oracle. Java Documentation. Dokumentasi mengenai class, object, ArrayList, Scanner, constructor, method, dan struktur kontrol Java.